Guía completa de condicionales en Python

Oct 27, 2026 · @nur

1. ¿Qué es un condicional y para qué sirve?

Un condicional permite que un programa tome una decisión: hace una cosa si algo es verdadero y otra cosa distinta si no lo es.

Las personas decidimos todo el día. Por ejemplo: «Si llueve, llevo paraguas; si no, salgo sin él». Python decide exactamente igual, pero con reglas muy exactas que hay que escribir sin errores.

En la vida diaria	En Python (idea)
Si llueve, llevo paraguas	`if llueve:` → llevar paraguas
Si tengo 18 años o más, puedo votar; si no, no puedo	`if edad >= 18:` → puede votar, `else:` → no puede
Si tengo dinero **y** tengo tiempo, voy al cine	`if dinero and tiempo:`
Si es sábado **o** domingo, descanso	`if dia == "sabado" or dia == "domingo":`

Cómo usar esta guía

1. Lee la explicación de cada tema y mira su diagrama.
2. Lee el enunciado de cada ejemplo e intenta resolverlo en tu cabeza antes de ver el código.
3. Sigue el análisis paso a paso, línea por línea, con el dedo o el cursor.
4. Escribe cada ejemplo en tu computador (no lo copies y pegues), ejecútalo y cambia los valores para ver qué pasa.
5. Al final, resuelve los 26 ejercicios sin mirar los ejemplos. Empiezan fáciles y suben poco a poco.

Dónde escribir y ejecutar el código

Puedes usar Google Colab, que es gratis y funciona desde el navegador sin instalar nada. También sirve IDLE, el editor que se instala junto con Python.

Qué verás en los ejemplos

- Un bloque gris con el **código** que debes escribir.
- Una línea que empieza con `#` es un **comentario**: Python la ignora y sirve para explicarte algo.
- Una **prueba de escritorio**: seguir el programa a mano, línea por línea, con valores concretos.
- La **salida**: lo que aparece en pantalla cuando se ejecuta el programa.

Mapa de la guía

Parte	Qué aprenderás
2. Conceptos previos	Variables, `print`, `input`, comparaciones y sangría

Parte	Qué aprenderás
3. `if` simple	Hacer algo solo cuando una condición es verdadera
4. `if` con `else`	Elegir entre dos caminos
5. Operador `and`	Exigir que se cumplan varias condiciones a la vez
6. Operador `or`	Aceptar que se cumpla al menos una condición
7. `and` y `or` juntos	Combinar ambos con paréntesis
8. Condicionales anidados	Una decisión dentro de otra decisión
9. Errores comunes y resumen	Qué falla más y una hoja de repaso
10 a 12. Ejercicios	8 básicos, 8 con `and`/`or` y 10 con anidación

2. Conceptos previos: lo mínimo para empezar

Antes de tomar decisiones necesitas seis herramientas: guardar datos, mostrar mensajes, pedir datos, hacer cuentas simples, comparar y respetar la sangría.

2.1 Variables: cajas con etiqueta

Una variable es una caja con nombre donde guardas un dato. Se crea con el signo `=`, que se lee «guarda lo de la derecha en el nombre de la izquierda».

```
edad = 20
nombre = "Ana"
nota = 4.5
```

Tipo de dato	Qué guarda	Ejemplo	Nombre en Python
Número entero	Números sin decimales	`20`	`int`
Número decimal	Números con punto decimal (se usa punto, no coma)	`4.5`	`float`
Texto	Letras y símbolos entre comillas	`"Ana"`	`str`
Verdadero o falso	Solo dos valores posibles	`True`, `False`	`bool`

Reglas para nombrar variables: sin espacios, sin empezar con número y, mejor, en minúsculas con guion bajo (`valor_total`).

2.2 `print`: mostrar mensajes en pantalla

`print` escribe en pantalla lo que va entre paréntesis. El texto va entre comillas; las variables, sin comillas. La coma separa las partes y deja un espacio entre ellas.

```
edad = 20
print("Hola")
print("Tu edad es", edad)
```

Salida en pantalla:

```
Hola
Tu edad es 20
```

2.3 `input`: pedir datos a la persona

`input` muestra una pregunta, espera que la persona escriba y presione Enter, y devuelve lo escrito. **Siempre lo devuelve como texto**, aunque la persona escriba un número. Por eso se convierte con `int()` (entero) o `float()` (decimal).

```
edad = int(input("¿Cuántos años tienes? "))
```

Cómo se ejecuta esa línea, de adentro hacia afuera:

1. `input(...)` muestra la pregunta «¿Cuántos años tienes?» y espera.
2. La persona escribe `20` y presiona Enter.
3. `input` entrega el **texto** `"20"`.
4. `int(...)` convierte ese texto en el **número** `20`.
5. `edad =` guarda el número 20 en la variable `edad`.

Quiero guardar	Escribo	Ejemplo
Un número entero	`int(input(...))`	`edad = int(input("Edad: "))`
Un número decimal	`float(input(...))`	`nota = float(input("Nota: "))`
Un texto	`input(...)`	`nombre = input("Nombre: ")`

> *Si no conviertes el número, Python no puede compararlo con otro número y se detiene con un error.*

2.4 Operaciones básicas

Operación	Símbolo	Ejemplo	Resultado
Suma	`+`	`5 + 3`	`8`
Resta	`-`	`5 - 3`	`2`
Multiplicación	`*`	`5 * 3`	`15`
División	`/`	`10 / 4`	`2.5` (la división siempre da decimal)

2.5 Comparaciones: preguntas que se responden con `True` o `False`

Una **condición** es una pregunta que Python responde solo con `True` (verdadero) o `False` (falso). Se arma comparando dos valores:

Símbolo	Se lee	Ejemplo	Resultado
`==`	es igual a	`5 == 5`	`True`

Símbolo	Se lee	Ejemplo	Resultado
`!=`	es distinto de	`5 != 3`	`True`
`>`	es mayor que	`5 > 8`	`False`
`<`	es menor que	`5 < 8`	`True`
`>=`	es mayor o igual que	`5 >= 5`	`True`
`<=`	es menor o igual que	`6 <= 5`	`False`

Tres detalles que confunden a casi todo el mundo:

- **`=` guarda y `==` pregunta.** `edad = 18` guarda el 18; `edad == 18` pregunta si la edad es 18.
- **El límite cuenta o no según el símbolo.** Si `edad` vale 18: `edad >= 18` es `True`, pero `edad > 18` es `False`.
- **En los textos importan las mayúsculas.** `"Ana" == "ana"` es `False`. En esta guía escribiremos los textos en minúsculas y sin tildes (`"sabado"`) para evitar errores.

2.6 Dos puntos y sangría

Python sabe qué líneas pertenecen a una decisión por dos señales:

1. La línea de la decisión **termina en dos puntos** (`:`).
2. Las líneas que dependen de ella van **corridas hacia la derecha, con 4 espacios** (la tecla Tab). Ese corrimiento se llama **sangría** o indentación, y el grupo de líneas con sangría se llama **bloque**.

```
if edad >= 18:
    print("Eres mayor de edad")   # con sangría: pertenece al if
    print("Puedes votar")         # con sangría: también pertenece al if
print("Fin del programa")         # sin sangría: NO pertenece al if
```

La última línea está pegada al borde izquierdo, así que se ejecuta **siempre**, sin importar la edad.

Vocabulario clave

Palabra	Significado
Condición	Pregunta cuya respuesta es `True` o `False`
Bloque	Líneas con sangría que dependen de un `if` o de un `else`
Ejecutar	Hacer que Python lea y realice las instrucciones
Prueba de escritorio	Seguir el programa a mano, línea por línea, con valores concretos

3. El `if` simple: hacer algo solo si se cumple una condición

`if` significa «si». Ejecuta un bloque de código únicamente cuando la condición es `True`; si es `False`, Python se lo salta y sigue con el resto del programa.

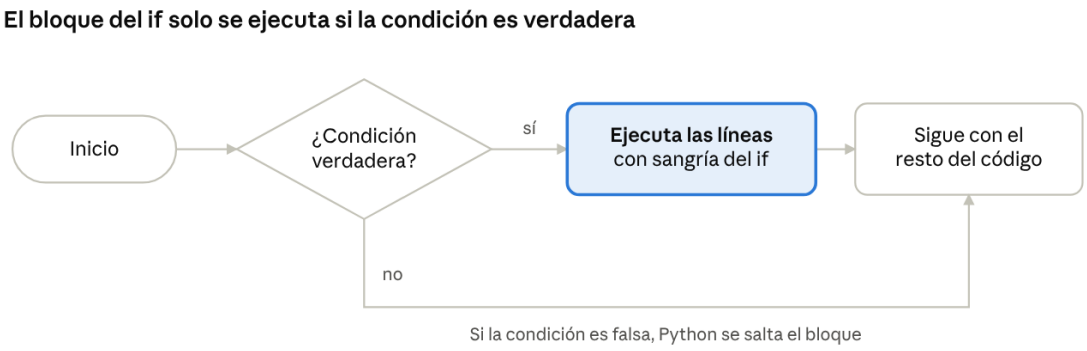
Estructura

```
if condicion:
    instruccion_que_se_ejecuta_solo_si_es_verdadera
```

Parte	Qué es	Ejemplo
`if`	Palabra reservada que significa «si». Siempre en minúscula	`if`
Condición	Una pregunta con respuesta `True` o `False`	`edad >= 18`
`:`	Dos puntos obligatorios al final de la línea	`:`
Bloque con sangría	Líneas con 4 espacios que se ejecutan solo si la condición es verdadera	`    print("Hola")`

Cómo decide Python

Python evalúa la condición. Si es verdadera entra al bloque; si es falsa se lo salta. En los dos casos, después continúa con la primera línea que no tiene sangría.



if simple · 1 decisión, 2 caminos

En el diagrama, «sí» significa que la condición dio `True` y «no» que dio `False`. Los dos caminos se vuelven a juntar al final.

Cinco ejemplos analizados paso a paso

Ejemplo 1: ¿Es mayor de edad?

Enunciado. Escribe un programa que pida la edad (número entero). Si la persona tiene 18 años o más, debe mostrar el mensaje `Eres mayor de edad`. Si tiene menos de 18, no debe mostrar nada.

```
edad = int(input("¿Cuántos años tienes? "))
if edad >= 18:
    print("Eres mayor de edad")
```

Análisis paso a paso, caso A: la persona escribe 20

1. Línea 1: Python pregunta la edad, recibe el texto `"20"`, lo convierte en el número 20 y lo guarda en `edad`.

2. Línea 2: Python evalúa `edad >= 18`, es decir, `20 >= 18`. La respuesta es `True`.
3. Como la condición es verdadera, Python **entra** al bloque con sangría.
4. Línea 3: se muestra `Eres mayor de edad`.
5. No hay más líneas: el programa termina.

Análisis paso a paso, caso B: la persona escribe 15

1. Línea 1: `edad` guarda el número 15.
2. Línea 2: Python evalúa `15 >= 18`. La respuesta es `False`.
3. Como la condición es falsa, Python **salta** la línea 3.
4. No hay más líneas: el programa termina sin mostrar nada.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`20`	`20 >= 18`	`True`	`Eres mayor de edad`
`18`	`18 >= 18`	`True`	`Eres mayor de edad`
`15`	`15 >= 18`	`False`	*(no muestra nada)*

> ***Idea clave:*** *el símbolo `>=` incluye el límite. Por eso quien tiene exactamente 18 años sí es mayor de edad.*

Ejemplo 2: Envío gratis

Enunciado. Escribe un programa que pida el valor total de una compra en pesos (número entero). Si el total es **mayor que** 50000, debe mostrar `Tu envío es gratis`. Si es 50000 o menos, no debe mostrar nada.

```
total = int(input("Valor total de la compra: "))
if total > 50000:
    print("Tu envío es gratis")
```

Análisis paso a paso, caso A: la persona escribe 60000

1. Línea 1: `total` guarda el número 60000.
2. Línea 2: Python evalúa `60000 > 50000`. La respuesta es `True`.
3. Línea 3: se muestra `Tu envío es gratis`.

Análisis paso a paso, caso B: la persona escribe 50000

1. Línea 1: `total` guarda el número 50000.
2. Línea 2: Python evalúa `50000 > 50000`. Un número **no es mayor que sí mismo**, así que la respuesta es `False`.
3. Python salta la línea 3 y el programa termina sin mostrar nada.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`60000`	`60000 > 50000`	`True`	`Tu envío es gratis`
`50000`	`50000 > 50000`	`False`	*(no muestra nada)*
`30000`	`30000 > 50000`	`False`	*(no muestra nada)*

> ***Idea clave:*** *`>` **no** incluye el límite y `>=` **sí** lo incluye. Una sola diferencia de símbolo cambia el resultado justo en el valor límite.*

Ejemplo 3: Clave de acceso

Enunciado. Un programa guarda la clave secreta `python123`. Debe pedir a la persona que escriba una clave. Si lo escrito es **exactamente igual** a la clave secreta, debe mostrar `Acceso concedido`. Si no es igual, no debe mostrar nada.

```
clave_secreta = "python123"
clave = input("Escribe la clave: ")
if clave == clave_secreta:
    print("Acceso concedido")
```

Análisis paso a paso, caso A: la persona escribe python123

1. Línea 1: la variable `clave_secreta` guarda el texto `"python123"`.
2. Línea 2: `input` devuelve el texto `python123` y se guarda en `clave`. No se usa `int()` porque es un texto, no un número.
3. Línea 3: Python evalúa `"python123" == "python123"`. Los dos textos son idénticos, así que es `True`.
4. Línea 4: se muestra `Acceso concedido`.

Análisis paso a paso, caso B: la persona escribe Python123 (con P mayúscula)

1. Línea 2: `clave` guarda el texto `"Python123"`.
2. Línea 3: Python compara `"Python123" == "python123"`. La `P` mayúscula y la `p` minúscula son letras distintas, así que es `False`.
3. Python salta la línea 4 y termina sin mostrar nada.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`python123`	`"python123" == "python123"`	`True`	`Acceso concedido`
`Python123`	`"Python123" == "python123"`	`False`	*(no muestra nada)*
`python`	`"python" == "python123"`	`False`	*(no muestra nada)*

> ***Idea clave:*** *`==` también compara textos, y distingue mayúsculas de minúsculas y cada carácter.*

Ejemplo 4: Aviso de frío

Enunciado. Escribe un programa que pida la temperatura en grados centígrados (número que puede tener decimales). Si la temperatura es **menor que** 15, debe mostrar `Hace frío: lleva chaqueta`. Si es 15 o más, no debe mostrar nada.

```
temperatura = float(input("¿Qué temperatura hace? "))
if temperatura < 15:
    print("Hace frío: lleva chaqueta")
```

Análisis paso a paso, caso A: la persona escribe 12.5

1. Línea 1: `float` convierte el texto `"12.5"` en el número decimal 12.5 y lo guarda en `temperatura`. Se usa `float` porque hay decimales.
2. Línea 2: Python evalúa `12.5 < 15`. Es `True`.
3. Línea 3: se muestra `Hace frío: lleva chaqueta`.

Análisis paso a paso, caso B: la persona escribe 15

1. Línea 1: `temperatura` guarda 15.0.
2. Línea 2: Python evalúa `15.0 < 15`. Son iguales, y «menor que» no incluye la igualdad, así que es `False`.
3. Python salta la línea 3 y termina sin mostrar nada.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`12.5`	`12.5 < 15`	`True`	`Hace frío: lleva chaqueta`
`15`	`15.0 < 15`	`False`	*(no muestra nada)*
`22`	`22.0 < 15`	`False`	*(no muestra nada)*

> ***Idea clave:*** *si la persona puede escribir decimales, convierte con `float()`. Con `int()` el programa fallaría al escribir `12.5`.*

Ejemplo 5: La sangría decide qué pertenece al `if`

Enunciado. Escribe un programa que pida una nota entre 0 y 5 (número que puede tener decimales). Si la nota es 3.0 o más, debe mostrar `Aprobaste` y, en otra línea, `¡Felicitaciones!`. Sin importar la nota, al final siempre debe mostrar `Fin del programa`.

```
nota = float(input("Escribe tu nota (0 a 5): "))
if nota >= 3.0:
    print("Aprobaste")
    print("¡Felicitaciones!")
print("Fin del programa")
```

Análisis paso a paso, caso A: la persona escribe 4.5

1. Línea 1: `nota` guarda 4.5.
2. Línea 2: Python evalúa `4.5 >= 3.0`. Es `True`, así que entra al bloque.
3. Línea 3 (con sangría): se muestra `Aprobaste`.
4. Línea 4 (con sangría): se muestra `¡Felicitaciones!`.
5. Línea 5 (**sin** sangría): ya no pertenece al `if`, se ejecuta siempre. Se muestra `Fin del programa`.

Análisis paso a paso, caso B: la persona escribe 2.0

1. Línea 1: `nota` guarda 2.0.
2. Línea 2: Python evalúa `2.0 >= 3.0`. Es `False`, así que salta **todo el bloque** (líneas 3 y 4).
3. Línea 5: está fuera del `if`, así que se ejecuta igual. Se muestra `Fin del programa`.

Entrada	Condición evaluada	Salida en pantalla
`4.5`	`4.5 >= 3.0` → `True`	`Aprobaste`, `¡Felicitaciones!` y `Fin del programa`

Entrada	Condición evaluada	Salida en pantalla
`3`	`3.0 >= 3.0` → `True`	`Aprobaste, ¡Felicitaciones!` y `Fin del programa`
`2.0`	`2.0 >= 3.0` → `False`	Solo `Fin del programa`

> ***Idea clave:*** *un `if` puede controlar varias líneas. Todas las que tengan sangría le pertenecen; la primera línea sin sangría marca dónde termina el bloque.*

4. `if` con `else`: elegir entre dos caminos

`else` significa «si no». Agrega un segundo camino que se ejecuta cuando la condición del `if` es falsa, así que el programa siempre hace una de las dos cosas: nunca ninguna y nunca las dos.

Es como una bifurcación en el camino: «¿Tengo sueño? Si sí, duermo; si no, estudio».

Estructura

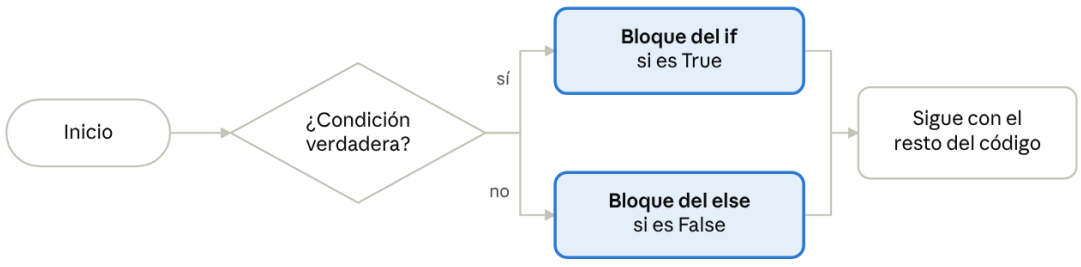
```
if condicion:
    # camino 1: se ejecuta si la condición es True
else:
    # camino 2: se ejecuta si la condición es False
```

Regla	Explicación
`else` va alineado con su `if`	Los dos quedan pegados al mismo borde izquierdo
`else` lleva dos puntos	Igual que el `if`: `else:`
`else` **no** lleva condición	Significa «en cualquier otro caso», sin preguntar nada
Solo se ejecuta un bloque	O el del `if` o el del `else`, nunca los dos
`else` nunca va solo	Siempre acompaña a un `if`

Cómo decide Python

Python evalúa la condición una sola vez. Si es `True` ejecuta el bloque del `if` y se salta el del `else`. Si es `False` hace lo contrario. Después continúa con el resto del programa.

Con else, el programa siempre toma uno de los dos caminos



Solo se ejecuta uno de los dos bloques

if con else · 1 decisión, 2 bloques

Observa que ambos caminos terminan en el mismo lugar: la primera línea sin sangría después del `else`.

Cinco ejemplos analizados paso a paso

Ejemplo 1: ¿Aprobó o reprobó?

Enunciado. Escribe un programa que pida una nota entre 0 y 5 (número que puede tener decimales). Si la nota es 3.0 o más, debe mostrar `Aprobaste`. En cualquier otro caso, debe mostrar `Reprobaste`.

```
nota = float(input("Escribe tu nota (0 a 5): "))
if nota >= 3.0:
    print("Aprobaste")
else:
    print("Reprobaste")
```

Análisis paso a paso, caso A: la persona escribe 4.2

1. Línea 1: `nota` guarda el número 4.2.
2. Línea 2: Python evalúa `4.2 >= 3.0`. Es `True`.
3. Línea 3: se ejecuta el bloque del `if` y se muestra `Aprobaste`.
4. Líneas 4 y 5: el bloque del `else` se **salta**, porque ya se ejecutó el otro camino.

Análisis paso a paso, caso B: la persona escribe 2.8

1. Línea 1: `nota` guarda el número 2.8.
2. Línea 2: Python evalúa `2.8 >= 3.0`. Es `False`.
3. Línea 3: el bloque del `if` se **salta**.
4. Línea 5: se ejecuta el bloque del `else` y se muestra `Reprobaste`.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`4.2`	`4.2 >= 3.0`	`True`	`Aprobaste`
`3.0`	`3.0 >= 3.0`	`True`	`Aprobaste`
`2.8`	`2.8 >= 3.0`	`False`	`Reprobaste`

> ***Idea clave:*** *con `if` y `else` siempre aparece exactamente un mensaje.*

Ejemplo 2: Tipo de entrada al cine

Enunciado. Escribe un programa que pida la edad (número entero). Si la persona tiene 12 años o más, debe mostrar `Entrada general`. Si tiene menos de 12 años, debe mostrar `Entrada infantil`.

```
edad = int(input("¿Cuántos años tienes? "))
if edad >= 12:
    print("Entrada general")
else:
    print("Entrada infantil")
```

Análisis paso a paso, caso A: la persona escribe 30

1. Línea 1: `edad` guarda el número 30.
2. Línea 2: Python evalúa `30 >= 12`. Es `True`.

3. Línea 3: se muestra `Entrada general`.
4. El bloque del `else` se salta.

Análisis paso a paso, caso B: la persona escribe 8

1. Línea 1: `edad` guarda el número 8.
2. Línea 2: Python evalúa `8 >= 12`. Es `False`.
3. El bloque del `if` se salta.
4. Línea 5: se muestra `Entrada infantil`.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`30`	`30 >= 12`	`True`	`Entrada general`
`12`	`12 >= 12`	`True`	`Entrada general`
`11`	`11 >= 12`	`False`	`Entrada infantil`

> ***Idea clave:*** *el valor límite (aquí el 12) se decide con el símbolo. Con `>=` el 12 va al `if`; con `>` iría al `else`.*

Ejemplo 3: Clave correcta o incorrecta

Enunciado. Un programa guarda la clave secreta `luna2026`. Debe pedir una clave a la persona. Si es exactamente igual a la secreta, debe mostrar `Clave correcta`. Si no es igual, debe mostrar `Clave incorrecta`.

```
clave_secreta = "luna2026"
clave = input("Escribe la clave: ")
if clave == clave_secreta:
    print("Clave correcta")
else:
    print("Clave incorrecta")
```

Análisis paso a paso, caso A: la persona escribe luna2026

1. Línea 1: `clave_secreta` guarda el texto `"luna2026"`.
2. Línea 2: `clave` guarda el texto `"luna2026"`.
3. Línea 3: Python evalúa `"luna2026" == "luna2026"`. Es `True`.
4. Línea 4: se muestra `Clave correcta`. El `else` se salta.

Análisis paso a paso, caso B: la persona escribe sol2026

1. Línea 2: `clave` guarda el texto `"sol2026"`.
2. Línea 3: Python evalúa `"sol2026" == "luna2026"`. Los textos son distintos, así que es `False`.
3. Línea 4 se salta.
4. Línea 6: se muestra `Clave incorrecta`.

Entrada	Condición evaluada	Resultado	Salida en pantalla
`luna2026`	`"luna2026" == "luna2026"`	`True`	`Clave correcta`
`sol2026`	`"sol2026" == "luna2026"`	`False`	`Clave incorrecta`

Entrada	Condición evaluada	Resultado	Salida en pantalla
`Luna2026`	`"Luna2026" == "luna2026"`	`False`	`Clave incorrecta`

> ***Idea clave:*** *la `L` mayúscula hace que el texto sea distinto. Para Python, una sola letra diferente basta para que `==` dé `False`.*

Ejemplo 4: Costo del envío

Enunciado. Escribe un programa que pida el valor de una compra en pesos (número entero). Si la compra es de 80000 o más, el envío cuesta 0. Si es menor de 80000, el envío cuesta 8000. El programa debe mostrar el costo del envío y el total a pagar (compra más envío), cada uno en una línea.

```
compra = int(input("Valor de la compra: "))
if compra >= 80000:
    envio = 0
else:
    envio = 8000
print("Envío:", envio)
print("Total a pagar:", compra + envio)
```

Análisis paso a paso, caso A: la persona escribe 90000

1. Línea 1: `compra` guarda 90000.
2. Línea 2: Python evalúa `90000 >= 80000`. Es `True`.
3. Línea 3: la variable `envio` guarda 0. El `else` se salta.
4. Línea 6 (sin sangría, se ejecuta siempre): muestra `Envío: 0`.
5. Línea 7: calcula `90000 + 0 = 90000` y muestra `Total a pagar: 90000`.

Análisis paso a paso, caso B: la persona escribe 50000

1. Línea 1: `compra` guarda 50000.
2. Línea 2: Python evalúa `50000 >= 80000`. Es `False`.
3. Línea 3 se salta. Línea 5: la variable `envio` guarda 8000.
4. Línea 6: muestra `Envío: 8000`.
5. Línea 7: calcula `50000 + 8000 = 58000` y muestra `Total a pagar: 58000`.

Entrada	Condición evaluada	Valor de `envio`	Salida en pantalla
`90000`	`90000 >= 80000` → `True`	`0`	`Envío: 0` y `Total a pagar: 90000`
`80000`	`80000 >= 80000` → `True`	`0`	`Envío: 0` y `Total a pagar: 80000`
`50000`	`50000 >= 80000` → `False`	`8000`	`Envío: 8000` y `Total a pagar: 58000`

> ***Idea clave:*** *dentro de un `if` o un `else` no solo se muestran mensajes: también se pueden guardar valores en variables y usarlos después, fuera del bloque.*

Ejemplo 5: El mayor de dos números

Enunciado. Escribe un programa que pida dos números enteros, uno por vez, y muestre el mayor de los dos con el formato `El mayor es:` seguido del número. Si los dos números son iguales, debe mostrar ese mismo número.

```
a = int(input("Primer número: "))
b = int(input("Segundo número: "))
if a > b:
    print("El mayor es:", a)
else:
    print("El mayor es:", b)
```

Análisis paso a paso, caso A: la persona escribe 7 y luego 3

1. Línea 1: `a` guarda 7. Línea 2: `b` guarda 3.
2. Línea 3: Python evalúa `7 > 3`. Es `True`.
3. Línea 4: se muestra `El mayor es: 7`. El `else` se salta.

Análisis paso a paso, caso B: la persona escribe 2 y luego 9

1. Línea 1: `a` guarda 2. Línea 2: `b` guarda 9.
2. Línea 3: Python evalúa `2 > 9`. Es `False`.
3. Línea 6: se muestra `El mayor es: 9`.

Análisis paso a paso, caso C: la persona escribe 5 y luego 5

1. `a` guarda 5 y `b` guarda 5.
2. Python evalúa `5 > 5`. Es `False` porque un número no es mayor que sí mismo.
3. Se ejecuta el `else` y se muestra `El mayor es: 5`, que es correcto.

Entrada (a, b)	Condición evaluada	Resultado	Salida en pantalla
`7`, `3`	`7 > 3`	`True`	`El mayor es: 7`
`2`, `9`	`2 > 9`	`False`	`El mayor es: 9`
`5`, `5`	`5 > 5`	`False`	`El mayor es: 5`

> ***Idea clave:*** *el `else` recoge* ***todo lo que no cumplió el `if`****, incluidos los empates. Antes de escribir un programa pregúntate qué debe pasar justo en el valor límite.*

5. El operador `and`: todas las condiciones deben cumplirse

`and` significa «y». Une dos o más condiciones, y el resultado es `True` solo si **todas** son verdaderas. Basta con que una sea falsa para que el resultado completo sea `False`.

Para entrar a una discoteca necesitas tener 18 años o más **y** llevar la cédula. Si te falta una de las dos cosas, no entras.

Tabla de verdad de `and`

Condición A (tengo 18 años o más)	Condición B (llevo la cédula)	`A and B`	Qué pasa en la discoteca
`True`	`True`	`True`	Entras
`True`	`False`	`False`	No entras
`False`	`True`	`False`	No entras
`False`	`False`	`False`	No entras

Solo hay **una** fila verdadera de cuatro: cuando las dos condiciones son verdaderas.

Estructura

```
if condicion1 and condicion2:
    # se ejecuta solo si las dos son True
else:
    # se ejecuta si al menos una es False
```

Regla de oro: cada lado del `and` es una condición completa

Cada condición debe escribirse entera, con su variable, su símbolo y su valor. No se puede «abreviar» repitiendo solo el símbolo.

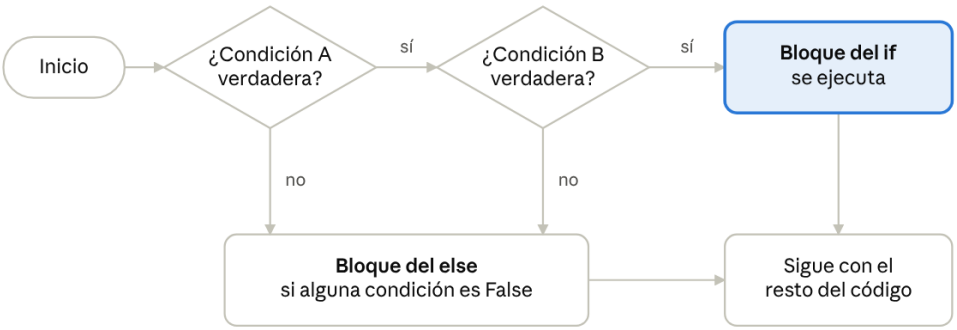
Incorrecto	Correcto
`if edad >= 18 and <= 60:`	`if edad >= 18 and edad <= 60:`
`if nota >= 0 and <= 5:`	`if nota >= 0 and nota <= 5:`

La primera forma produce un `SyntaxError`, porque `<= 60` por sí solo no es una pregunta completa.

Cómo decide Python

Python evalúa las condiciones de izquierda a derecha. Si la primera es `False`, ya sabe que el resultado final será `False` y ni siquiera mira las demás. Si es `True`, revisa la siguiente, y así hasta el final.

Con and, todas las condiciones deben ser verdaderas



Si A es falsa, Python ni siquiera revisa B

and · 2 condiciones en cadena

Para llegar al bloque del `if` hay que pasar por dos filtros seguidos. Fallar en cualquiera de los dos manda al `else`.

Cinco ejemplos analizados paso a paso

Ejemplo 1: Entrada a un evento

Enunciado. Escribe un programa que pida la edad (número entero) y que pregunte si la persona lleva la cédula (se escribe `si` o `no`, en minúsculas y sin tilde). Si tiene 18 años o más **y** lleva la cédula, debe mostrar `Puedes entrar`. En cualquier otro caso, debe mostrar `No puedes entrar`.

```
edad = int(input("¿Cuántos años tienes? "))
cedula = input("¿Llevas la cédula? (si/no): ")
if edad >= 18 and cedula == "si":
    print("Puedes entrar")
else:
    print("No puedes entrar")
```

Análisis paso a paso, caso A: la persona escribe 20 y luego si

1. Línea 1: `edad` guarda 20. Línea 2: `cedula` guarda el texto `"si"`.
2. Línea 3: Python evalúa la primera parte: `20 >= 18` es `True`.
3. Como la primera fue verdadera, evalúa la segunda: `"si" == "si"` es `True`.
4. `True and True` da `True`. Se ejecuta el bloque del `if` y se muestra `Puedes entrar`.

Análisis paso a paso, caso B: la persona escribe 20 y luego no

1. `edad` guarda 20 y `cedula` guarda `"no"`.
2. Primera parte: `20 >= 18` es `True`.
3. Segunda parte: `"no" == "si"` es `False`.
4. `True and False` da `False`. Se ejecuta el `else` y se muestra `No puedes entrar`.

Análisis paso a paso, caso C: la persona escribe 16 y luego si

1. `edad` guarda 16 y `cedula` guarda `"si"`.

2. Primera parte: `16 >= 18` es `False`.
3. Python ya sabe que `False and …` dará `False`, así que no necesita revisar la cédula.
4. Se ejecuta el `else` y se muestra `No puedes entrar`.

Edad	Cédula	`edad >= 18`	`cedula == "si"`	Resultado con `and`	Salida
20	si	True	True	True	Puedes entrar
20	no	True	False	False	No puedes entrar
16	si	False	True	False	No puedes entrar
16	no	False	False	False	No puedes entrar

> ***Idea clave:*** *con `and`, una sola condición falsa cierra la puerta.*

Ejemplo 2: ¿La nota está en el rango válido?

Enunciado. Escribe un programa que pida una nota (número que puede tener decimales). Una nota es válida si está entre 0 y 5, **incluidos** el 0 y el 5. Si es válida, debe mostrar `Nota válida`; si no, debe mostrar `Nota no válida`.

```
nota = float(input("Escribe la nota: "))
if nota >= 0 and nota <= 5:
    print("Nota válida")
else:
    print("Nota no válida")
```

Análisis paso a paso, caso A: la persona escribe 4.0

1. `nota` guarda 4.0.
2. Primera parte: `4.0 >= 0` es `True`.
3. Segunda parte: `4.0 <= 5` es `True`.
4. `True and True` da `True`. Se muestra `Nota válida`.

Análisis paso a paso, caso B: la persona escribe 7

1. `nota` guarda 7.0.
2. Primera parte: `7.0 >= 0` es `True`.
3. Segunda parte: `7.0 <= 5` es `False`.
4. `True and False` da `False`. Se muestra `Nota no válida`.

Análisis paso a paso, caso C: la persona escribe -1

1. `nota` guarda -1.0.
2. Primera parte: `-1.0 >= 0` es `False`. Python ya no revisa la segunda.
3. El resultado es `False`. Se muestra `Nota no válida`.

Entrada	`nota >= 0`	`nota <= 5`	Resultado con `and`	Salida
4.0	True	True	True	Nota válida
5	True	True	True	Nota válida

Entrada	`nota >= 0`	`nota <= 5`	Resultado con `and`	Salida
`0`	`True`	`True`	`True`	`Nota válida`
`7`	`True`	`False`	`False`	`Nota no válida`
`-1`	`False`	`True`	`False`	`Nota no válida`

> ***Idea clave:*** *un rango («entre un mínimo y un máximo») siempre se escribe con `and`: una condición para cada extremo.*

Ejemplo 3: ¿Aprobó el curso?

Enunciado. Para aprobar un curso se necesitan **dos** requisitos: una nota de 3.0 o más **y** una asistencia de 80 o más (porcentaje, número entero). Escribe un programa que pida la nota (puede tener decimales) y la asistencia, en ese orden. Si cumple los dos requisitos, debe mostrar `Aprobaste el curso`; si no, debe mostrar `No aprobaste el curso`.

```
nota = float(input("Nota final: "))
asistencia = int(input("Porcentaje de asistencia: "))
if nota >= 3.0 and asistencia >= 80:
    print("Aprobaste el curso")
else:
    print("No aprobaste el curso")
```

Análisis paso a paso, caso A: la persona escribe 4.0 y luego 90

1. `nota` guarda 4.0 y `asistencia` guarda 90.
2. Primera parte: `4.0 >= 3.0` es `True`.
3. Segunda parte: `90 >= 80` es `True`.
4. `True and True` da `True`. Se muestra `Aprobaste el curso`.

Análisis paso a paso, caso B: la persona escribe 4.0 y luego 70

1. `nota` guarda 4.0 y `asistencia` guarda 70.
2. Primera parte: `4.0 >= 3.0` es `True`.
3. Segunda parte: `70 >= 80` es `False`.
4. `True and False` da `False`. Se muestra `No aprobaste el curso`, aunque la nota sea buena.

Análisis paso a paso, caso C: la persona escribe 2.5 y luego 95

1. `nota` guarda 2.5 y `asistencia` guarda 95.
2. Primera parte: `2.5 >= 3.0` es `False`. Python ya no revisa la asistencia.
3. El resultado es `False`. Se muestra `No aprobaste el curso`, aunque la asistencia sea excelente.

Nota	Asistencia	`nota >= 3.0`	`asistencia >= 80`	Resultado con `and`	Salida
`4.0`	`90`	`True`	`True`	`True`	`Aprobaste el curso`
`3.0`	`80`	`True`	`True`	`True`	`Aprobaste el curso`
`4.0`	`70`	`True`	`False`	`False`	`No aprobaste el curso`

Nota	Asistencia	`nota >= 3.0`	`asistencia >= 80`	Resultado con `and`	Salida
2.5	95	`False`	`True`	`False`	`No aprobaste el curso`

> ***Idea clave:*** *`and` sirve para **requisitos**: todos son obligatorios.*

Ejemplo 4: Retiro en un cajero

Enunciado. Escribe un programa que pida el saldo disponible y el valor que la persona quiere retirar (ambos enteros, en pesos). El retiro se permite solo si el valor es **mayor que 0 y** no supera el saldo (es menor o igual al saldo). Si se permite, debe mostrar `Retiro realizado` y, en otra línea, `Nuevo saldo:` seguido del saldo que queda. Si no se permite, debe mostrar `Retiro no permitido`.

```
saldo = int(input("Saldo disponible: "))
retiro = int(input("Valor a retirar: "))
if retiro > 0 and retiro <= saldo:
    print("Retiro realizado")
    print("Nuevo saldo:", saldo - retiro)
else:
    print("Retiro no permitido")
```

Análisis paso a paso, caso A: la persona escribe 200000 y luego 50000

1. `saldo` guarda 200000 y `retiro` guarda 50000.
2. Primera parte: `50000 > 0` es `True`.
3. Segunda parte: `50000 <= 200000` es `True`.
4. `True and True` da `True`. Se muestra `Retiro realizado`.
5. Se calcula `200000 - 50000 = 150000` y se muestra `Nuevo saldo: 150000`.

Análisis paso a paso, caso B: la persona escribe 200000 y luego 250000

1. `saldo` guarda 200000 y `retiro` guarda 250000.
2. Primera parte: `250000 > 0` es `True`.
3. Segunda parte: `250000 <= 200000` es `False`.
4. `True and False` da `False`. Se muestra `Retiro no permitido`.

Análisis paso a paso, caso C: la persona escribe 200000 y luego 0

1. `retiro` guarda 0.
2. Primera parte: `0 > 0` es `False`. Python ya no revisa la segunda.
3. Se muestra `Retiro no permitido`, porque retirar 0 pesos no tiene sentido.

Saldo	Retiro	`retiro > 0`	`retiro <= saldo`	Resultado con `and`	Salida
`200000`	`50000`	`True`	`True`	`True`	`Retiro realizado` y `Nuevo saldo: 150000`
`200000`	`200000`	`True`	`True`	`True`	`Retiro realizado` y `Nuevo saldo: 0`
`200000`	`250000`	`True`	`False`	`False`	`Retiro no permitido`

Saldo	Retiro	`retiro > 0`	`retiro <= saldo`	Resultado con `and`	Salida
`200000`	`0`	`False`	`True`	`False`	`Retiro no permitido`

> ***Idea clave:*** *dentro del bloque del `if` puedes hacer una resta (o cualquier cuenta) y mostrar el resultado.*

Ejemplo 5: Tres condiciones con `and`

Enunciado. Para subir a una atracción se exigen **tres** requisitos: tener 10 años o más, medir 120 centímetros o más y haber pagado el tiquete. Escribe un programa que pida, en este orden, la edad (entero), la estatura en centímetros (entero) y si pagó el tiquete (`si` o `no`, en minúsculas y sin tilde). Si cumple los tres requisitos, debe mostrar `Puedes subir`; si no, debe mostrar `No puedes subir`.

```
edad = int(input("Edad: "))
estatura = int(input("Estatura en cm: "))
pago = input("¿Pagaste el tiquete? (si/no): ")
if edad >= 10 and estatura >= 120 and pago == "si":
    print("Puedes subir")
else:
    print("No puedes subir")
```

Análisis paso a paso, caso A: la persona escribe 12, 130 y si

1. `edad` guarda 12, `estatura` guarda 130 y `pago` guarda `"si"`.
2. Primera parte: `12 >= 10` es `True`.
3. Segunda parte: `130 >= 120` es `True`.
4. Tercera parte: `"si" == "si"` es `True`.
5. `True and True and True` da `True`. Se muestra `Puedes subir`.

Análisis paso a paso, caso B: la persona escribe 12, 110 y si

1. Primera parte: `12 >= 10` es `True`.
2. Segunda parte: `110 >= 120` es `False`. Python se detiene aquí.
3. El resultado completo es `False`. Se muestra `No puedes subir`.

Edad	Estatura	Pago	Primera	Segunda	Tercera	Resultado	Salida
`12`	`130`	`si`	`True`	`True`	`True`	`True`	`Puedes subir`
`12`	`110`	`si`	`True`	`False`	*(no se revisa)*	`False`	`No puedes subir`
`8`	`130`	`si`	`False`	*(no se revisa)*	*(no se revisa)*	`False`	`No puedes subir`
`12`	`130`	`no`	`True`	`True`	`False`	`False`	`No puedes subir`

> ***Idea clave:*** *puedes unir tantas condiciones como necesites con `and`. Todas deben ser verdaderas.*

6. El operador `or`: basta con que se cumpla una

`or` significa «o». Une dos o más condiciones, y el resultado es `True` cuando **al menos una** es verdadera. Solo es `False` si **todas** son falsas.

Para desayunar me sirve pan **o** arepa. Si hay una de las dos, desayuno; solo me quedo sin desayunar si no hay ninguna.

Tabla de verdad de `or`

Condición A (hay pan)	Condición B (hay arepa)	`A or B`	Qué pasa en el desayuno
`True`	`True`	`True`	Desayuno
`True`	`False`	`True`	Desayuno
`False`	`True`	`True`	Desayuno
`False`	`False`	`False`	No desayuno

Solo hay **una** fila falsa de cuatro: cuando las dos condiciones son falsas. Fíjate que el `or` también da `True` si se cumplen las dos a la vez.

Estructura

```
if condicion1 or condicion2:
    # se ejecuta si al menos una es True
else:
    # se ejecuta solo si las dos son False
```

Comparación rápida: `and` frente a `or`

	`and` («y»)	`or` («o»)
Idea	Se deben cumplir **todas**	Basta con **una**
Es `True` cuando…	Todas son `True`	Al menos una es `True`
Es `False` cuando…	Al menos una es `False`	Todas son `False`
Sirve para…	Requisitos obligatorios y rangos	Alternativas y opciones

El error más frecuente con `or`

Cada lado del `or` también debe ser una condición completa.

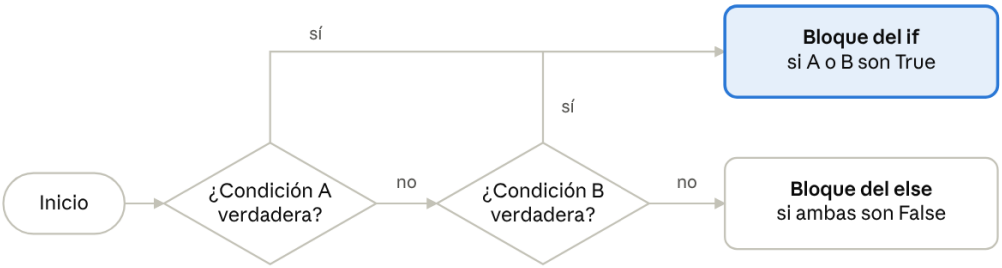
Incorrecto (siempre da `True`)	Correcto
`if dia == "sabado" or "domingo":`	`if dia == "sabado" or dia == "domingo":`

La forma incorrecta **no muestra ningún error**, pero el programa se equivoca: Python lee `"domingo"` solo, como si fuera una condición, y cualquier texto que no esté vacío cuenta como verdadero. Resultado: siempre dirá que es fin de semana.

Cómo decide Python

Python evalúa de izquierda a derecha. Si la primera condición es `True`, ya sabe que el resultado será `True` y no revisa las demás. Si es `False`, revisa la siguiente.

Con or, basta con que una condición sea verdadera



Si A ya es verdadera, Python ni siquiera revisa B

or · 2 condiciones, dos caminos al bloque

Aquí hay dos caminos para llegar al bloque del `if`. Solo se llega al `else` si se fallan las dos condiciones.

Cinco ejemplos analizados paso a paso

Ejemplo 1: ¿Es fin de semana?

Enunciado. Escribe un programa que pida el nombre de un día de la semana (en minúsculas y sin tilde: `lunes`, `martes`, `miercoles`, `jueves`, `viernes`, `sabado`, `domingo`). Si el día es `sabado` o `domingo`, debe mostrar `Es fin de semana`. En cualquier otro caso, debe mostrar `Es día de semana`.

```
dia = input("¿Qué día es hoy? ")
if dia == "sabado" or dia == "domingo":
    print("Es fin de semana")
else:
    print("Es día de semana")
```

Análisis paso a paso, caso A: la persona escribe sabado

1. `dia` guarda el texto `"sabado"`.
2. Primera parte: `"sabado" == "sabado"` es `True`.
3. Como con `or` basta una verdadera, Python ya no revisa la segunda parte.
4. El resultado es `True`. Se muestra `Es fin de semana`.

Análisis paso a paso, caso B: la persona escribe domingo

1. `dia` guarda el texto `"domingo"`.
2. Primera parte: `"domingo" == "sabado"` es `False`. Python debe seguir.
3. Segunda parte: `"domingo" == "domingo"` es `True`.
4. `False or True` da `True`. Se muestra `Es fin de semana`.

Análisis paso a paso, caso C: la persona escribe lunes

1. `dia` guarda el texto `"lunes"`.
2. Primera parte: `"lunes" == "sabado"` es `False`.
3. Segunda parte: `"lunes" == "domingo"` es `False`.
4. `False or False` da `False`. Se ejecuta el `else` y se muestra `Es día de semana`.

Entrada	`dia == "sabado"`	`dia == "domingo"`	Resultado con `or`	Salida
`sabado`	`True`	*(no se revisa)*	`True`	`Es fin de semana`
`domingo`	`False`	`True`	`True`	`Es fin de semana`
`lunes`	`False`	`False`	`False`	`Es día de semana`

> ***Idea clave:*** *`or` sirve cuando hay varias opciones válidas para una misma variable.*

Ejemplo 2: Ingreso a una sala especial

Enunciado. Escribe un programa que pregunte si la persona tiene invitación y si es socia del club (cada respuesta es `si` o `no`, en minúsculas y sin tilde), en ese orden. Si tiene invitación **o** es socia (o ambas cosas), debe mostrar `Bienvenido`. Si no cumple ninguna, debe mostrar `Acceso denegado`.

```
invitacion = input("¿Tienes invitación? (si/no): ")
socio = input("¿Eres socia del club? (si/no): ")
if invitacion == "si" or socio == "si":
    print("Bienvenido")
else:
    print("Acceso denegado")
```

Análisis paso a paso, caso A: la persona escribe si y luego no

1. `invitacion` guarda `"si"` y `socio` guarda `"no"`.
2. Primera parte: `"si" == "si"` es `True`. Python ya no revisa la segunda.
3. Se muestra `Bienvenido`.

Análisis paso a paso, caso B: la persona escribe no y luego si

1. `invitacion` guarda `"no"` y `socio` guarda `"si"`.
2. Primera parte: `"no" == "si"` es `False`.
3. Segunda parte: `"si" == "si"` es `True`.
4. `False or True` da `True`. Se muestra `Bienvenido`.

Análisis paso a paso, caso C: la persona escribe no y luego no

1. Las dos comparaciones dan `False`.
2. `False or False` da `False`. Se muestra `Acceso denegado`.

Invitación	Socia	Primera	Segunda	Resultado con `or`	Salida
`si`	`no`	`True`	*(no se revisa)*	`True`	`Bienvenido`
`no`	`si`	`False`	`True`	`True`	`Bienvenido`
`si`	`si`	`True`	*(no se revisa)*	`True`	`Bienvenido`

Invitación	Socia	Primera	Segunda	Resultado con `or`	Salida
`no`	`no`	`False`	`False`	`False`	`Acceso denegado`

> ***Idea clave:*** *con `or`, cumplir las dos cosas también sirve; no hace falta elegir solo una.*

Ejemplo 3: Nota fuera de rango

Enunciado. Escribe un programa que pida una nota (número que puede tener decimales). Si la nota es menor que 0 **o** mayor que 5, debe mostrar `Nota no válida`. En cualquier otro caso (incluidos el 0 y el 5), debe mostrar `Nota válida`.

```
nota = float(input("Escribe la nota: "))
if nota < 0 or nota > 5:
    print("Nota no válida")
else:
    print("Nota válida")
```

Análisis paso a paso, caso A: la persona escribe -1

1. `nota` guarda -1.0.
2. Primera parte: `-1.0 < 0` es `True`. Con `or` ya es suficiente, así que Python no revisa la segunda.
3. Se muestra `Nota no válida`.

Análisis paso a paso, caso B: la persona escribe 7

1. `nota` guarda 7.0.
2. Primera parte: `7.0 < 0` es `False`. Python debe seguir.
3. Segunda parte: `7.0 > 5` es `True`.
4. `False or True` da `True`. Se muestra `Nota no válida`.

Análisis paso a paso, caso C: la persona escribe 4.0

1. `nota` guarda 4.0.
2. Primera parte: `4.0 < 0` es `False`.
3. Segunda parte: `4.0 > 5` es `False`.
4. `False or False` da `False`. Se ejecuta el `else` y se muestra `Nota válida`.

Entrada	`nota < 0`	`nota > 5`	Resultado con `or`	Salida
`-1`	`True`	*(no se revisa)*	`True`	`Nota no válida`
`7`	`False`	`True`	`True`	`Nota no válida`
`4.0`	`False`	`False`	`False`	`Nota válida`
`0`	`False`	`False`	`False`	`Nota válida`
`5`	`False`	`False`	`False`	`Nota válida`

> ***Idea clave:*** *este ejemplo resuelve el mismo problema que el Ejemplo 2 de `and`, pero «al revés». Con `and` preguntamos si está **dentro** del rango (`>= 0 and <= 5`); con `or` preguntamos si está **fuera** (`< 0 or > 5`). Para valores **fuera** de un rango se usa `or`.*

Ejemplo 4: Tarifa reducida

Enunciado. Un parque cobra tarifa reducida a niños menores de 12 años **o** a personas de 65 años o más. Escribe un programa que pida la edad (entero). Si la persona cumple alguna de las dos condiciones, debe mostrar `Tarifa reducida`; si no, debe mostrar `Tarifa completa`.

```
edad = int(input("¿Cuántos años tienes? "))
if edad < 12 or edad >= 65:
    print("Tarifa reducida")
else:
    print("Tarifa completa")
```

Análisis paso a paso, caso A: la persona escribe 8

1. `edad` guarda 8.
2. Primera parte: `8 < 12` es `True`. Python ya no revisa la segunda.
3. Se muestra `Tarifa reducida`.

Análisis paso a paso, caso B: la persona escribe 70

1. `edad` guarda 70.
2. Primera parte: `70 < 12` es `False`.
3. Segunda parte: `70 >= 65` es `True`.
4. `False or True` da `True`. Se muestra `Tarifa reducida`.

Análisis paso a paso, caso C: la persona escribe 30

1. `edad` guarda 30.
2. Primera parte: `30 < 12` es `False`.
3. Segunda parte: `30 >= 65` es `False`.
4. `False or False` da `False`. Se muestra `Tarifa completa`.

Edad	`edad < 12`	`edad >= 65`	Resultado con `or`	Salida
8	`True`	*(no se revisa)*	`True`	`Tarifa reducida`
11	`True`	*(no se revisa)*	`True`	`Tarifa reducida`
12	`False`	`False`	`False`	`Tarifa completa`
30	`False`	`False`	`False`	`Tarifa completa`
65	`False`	`True`	`True`	`Tarifa reducida`
70	`False`	`True`	`True`	`Tarifa reducida`

> ***Idea clave:*** *una persona nunca puede ser a la vez menor de 12 y mayor de 65, pero eso no importa:* `or` *solo necesita que se cumpla* ***una*** *de las dos.*

Ejemplo 5: Envío gratis con tres opciones

Enunciado. Una tienda ofrece envío gratis si se cumple **al menos una** de estas tres opciones: la compra es de 100000 pesos o más, la persona es cliente premium, o la persona tiene un cupón. Escribe un programa que pida, en este orden, el valor de la compra (entero), si es cliente premium (`si` o `no`) y si

tiene cupón (`si` o `no`). Si cumple alguna opción, debe mostrar `Envío gratis`; si no, debe mostrar `Envío con costo`.

```
compra = int(input("Valor de la compra: "))
premium = input("¿Eres cliente premium? (si/no): ")
cupon = input("¿Tienes cupón? (si/no): ")
if compra >= 100000 or premium == "si" or cupon == "si":
    print("Envío gratis")
else:
    print("Envío con costo")
```

Análisis paso a paso, caso A: la persona escribe 120000, no y no

1. `compra` guarda 120000, `premium` guarda `"no"` y `cupon` guarda `"no"`.
2. Primera parte: `120000 >= 100000` es `True`. Python no revisa las otras dos.
3. Se muestra `Envío gratis`.

Análisis paso a paso, caso B: la persona escribe 40000, no y si

1. `compra` guarda 40000, `premium` guarda `"no"` y `cupon` guarda `"si"`.
2. Primera parte: `40000 >= 100000` es `False`.
3. Segunda parte: `"no" == "si"` es `False`.
4. Tercera parte: `"si" == "si"` es `True`.
5. `False or False or True` da `True`. Se muestra `Envío gratis`.

Análisis paso a paso, caso C: la persona escribe 40000, no y no

1. Primera parte: `40000 >= 100000` es `False`.
2. Segunda parte: `"no" == "si"` es `False`.
3. Tercera parte: `"no" == "si"` es `False`.
4. `False or False or False` da `False`. Se muestra `Envío con costo`.

Compra	Premium	Cupón	Primera	Segunda	Tercera	Resultado	Salida
`120000`	`no`	`no`	`True`	*(no se revisa)*	*(no se revisa)*	`True`	`Envío gratis`
`40000`	`si`	`no`	`False`	`True`	*(no se revisa)*	`True`	`Envío gratis`
`40000`	`no`	`si`	`False`	`False`	`True`	`True`	`Envío gratis`
`40000`	`no`	`no`	`False`	`False`	`False`	`False`	`Envío con costo`

> ***Idea clave:*** *igual que con `and`, puedes encadenar tantas condiciones como necesites con `or`. Basta con que una sea verdadera.*

7. `and` y `or` juntos: cuándo usar paréntesis

Puedes mezclar `and` y `or` en una misma condición, pero Python les da distinta prioridad, y por eso casi siempre necesitas paréntesis para que el programa haga lo que tú quieres.

La regla de prioridad

En matemáticas, `2 + 3 * 4` da 14 porque la multiplicación se hace antes que la suma. Con las condiciones pasa algo parecido: **`and` se resuelve antes que `or`**.

Orden	Qué resuelve Python primero
1.º	Lo que está entre **paréntesis** `( )`
2.º	Los **`and`**
3.º	Los **`or`**

Por qué importa: un mismo texto, dos significados

Queremos esta regla: «Puedes entrar si eres mayor de edad **y** (tienes boleta **o** eres invitado)».

Cómo lo escribo	Cómo lo lee Python	¿Qué pasa con un menor invitado?
`edad >= 18 and boleta == "si" or invitado == "si"`	`(edad >= 18 and boleta == "si") or invitado == "si"`	**Entra** (error: el `or` se evalúa al final y el invitado pasa solo)
`edad >= 18 and (boleta == "si" or invitado == "si")`	Primero el paréntesis, luego el `and`	**No entra** (correcto)

> ***Recomendación:*** *usa paréntesis siempre que mezcles `and` con `or`, aunque Python no los necesite. Hacen el código más claro y evitan errores.*

Estructura

```
if condicion1 and (condicion2 or condicion3):
    # se ejecuta si 1 es True Y además al menos una de 2 o 3 es True
else:
    # se ejecuta en cualquier otro caso
```

Tabla de verdad de `A and (B or C)`

Para que la condición completa sea `True` se necesita que A sea verdadera y, además, que B o C (o ambas) lo sea.

A	B	C	`B or C`	`A and (B or C)`
`True`	`True`	`True`	`True`	**`True`**
`True`	`True`	`False`	`True`	**`True`**
`True`	`False`	`True`	`True`	**`True`**
`True`	`False`	`False`	`False`	`False`
`False`	`True`	`True`	`True`	`False`
`False`	`True`	`False`	`True`	`False`

A	B	C	B or C	A and (B or C)
False	False	True	True	False
False	False	False	False	False

Cómo evaluar una condición mezclada

1. Resuelve cada comparación y escribe su `True` o `False`.
2. Resuelve lo que está **dentro del paréntesis**.
3. Resuelve los `and`.
4. Resuelve los `or`.
5. El último valor que queda es el resultado.

Cinco ejemplos analizados paso a paso

Ejemplo 1: Evento con boleta o invitación

Enunciado. Para entrar a un evento hay que tener 18 años o más **y**, además, tener boleta **o** ser invitado. Escribe un programa que pida, en este orden, la edad (entero), si tiene boleta (`si` o `no`) y si es invitado (`si` o `no`). Si puede entrar, debe mostrar `Puedes entrar`; si no, debe mostrar `No puedes entrar`.

```
edad = int(input("Edad: "))
boleta = input("¿Tienes boleta? (si/no): ")
invitado = input("¿Eres invitado? (si/no): ")
if edad >= 18 and (boleta == "si" or invitado == "si"):
    print("Puedes entrar")
else:
    print("No puedes entrar")
```

Análisis paso a paso, caso A: la persona escribe 20, si y no

1. `edad` guarda 20, `boleta` guarda `"si"` e `invitado` guarda `"no"`.
2. Comparaciones: `20 >= 18` es `True`; `"si" == "si"` es `True`; `"no" == "si"` es `False`.
3. Paréntesis: `True or False` da `True`.
4. Cálculo final: `True and True` da `True`. Se muestra `Puedes entrar`.

Análisis paso a paso, caso B: la persona escribe 20, no y no

1. Comparaciones: `20 >= 18` es `True`; `"no" == "si"` es `False`; `"no" == "si"` es `False`.
2. Paréntesis: `False or False` da `False`.
3. Cálculo final: `True and False` da `False`. Se muestra `No puedes entrar`.

Análisis paso a paso, caso C: la persona escribe 16, si y si

1. Comparaciones: `16 >= 18` es `False`; las otras dos son `True`.
2. Paréntesis: `True or True` da `True`.
3. Cálculo final: `False and True` da `False`. Se muestra `No puedes entrar`: no importa cuántas boletas tenga, el menor no cumple la edad.

Edad	Boleta	Invitado	`edad >= 18`	`boleta or invitado` (paréntesis)	Resultado final	Salida
`20`	`si`	`no`	`True`	`True`	`True`	`Puedes entrar`
`20`	`no`	`si`	`True`	`True`	`True`	`Puedes entrar`
`20`	`no`	`no`	`True`	`False`	`False`	`No puedes entrar`
`16`	`si`	`si`	`False`	`True`	`False`	`No puedes entrar`

> ***Idea clave:*** *el paréntesis agrupa las opciones («boleta o invitado») y el* `and` *exige que ese grupo se cumpla* ***junto con*** *la edad.*

Ejemplo 2: Beca con dos caminos

Enunciado. Una beca se otorga a quien tenga un promedio de 4.0 o más **y** cumpla al menos una de estas condiciones: pertenecer al estrato 1 o 2, o ser deportista. Escribe un programa que pida, en este orden, el promedio (puede tener decimales), el estrato (entero del 1 al 6) y si es deportista (`si` o `no`). Si recibe la beca, debe mostrar `Obtienes la beca`; si no, debe mostrar `No obtienes la beca`.

```
promedio = float(input("Promedio: "))
estrato = int(input("Estrato (1 a 6): "))
deportista = input("¿Eres deportista? (si/no): ")
if promedio >= 4.0 and (estrato <= 2 or deportista == "si"):
    print("Obtienes la beca")
else:
    print("No obtienes la beca")
```

Análisis paso a paso, caso A: la persona escribe 4.5, 2 y no

1. Comparaciones: `4.5 >= 4.0` es `True`; `2 <= 2` es `True`; `"no" == "si"` es `False`.
2. Paréntesis: `True or False` da `True`.
3. Cálculo final: `True and True` da `True`. Se muestra `Obtienes la beca`.

Análisis paso a paso, caso B: la persona escribe 4.5, 5 y si

1. Comparaciones: `4.5 >= 4.0` es `True`; `5 <= 2` es `False`; `"si" == "si"` es `True`.
2. Paréntesis: `False or True` da `True`.
3. Cálculo final: `True and True` da `True`. Se muestra `Obtienes la beca`.

Análisis paso a paso, caso C: la persona escribe 4.5, 4 y no

1. Comparaciones: `4.5 >= 4.0` es `True`; `4 <= 2` es `False`; `"no" == "si"` es `False`.
2. Paréntesis: `False or False` da `False`.
3. Cálculo final: `True and False` da `False`. Se muestra `No obtienes la beca`.

Promedio	Estrato	Deportista	`promedio >= 4.0`	`estrato <= 2 or deportista == "si"`	Resultado final	Salida
`4.5`	`2`	`no`	`True`	`True`	`True`	`Obtienes la beca`
`4.5`	`5`	`si`	`True`	`True`	`True`	`Obtienes la beca`
`4.5`	`4`	`no`	`True`	`False`	`False`	`No obtienes la beca`
`3.5`	`1`	`si`	`False`	`True`	`False`	`No obtienes la beca`

> ***Idea clave:*** *un requisito obligatorio (promedio) se une con `and` a un grupo de alternativas (estrato o deporte) encerrado entre paréntesis.*

Ejemplo 3: Entrada gratis al museo

Enunciado. Un museo da entrada gratis a quienes tengan menos de 12 años **o** 65 años o más, pero **solo si** viven en la ciudad. Escribe un programa que pida la edad (entero) y si la persona es residente (`si` o `no`), en ese orden. Si recibe entrada gratis, debe mostrar `Entrada gratis`; si no, debe mostrar `Paga entrada`.

```
edad = int(input("Edad: "))
residente = input("¿Vives en la ciudad? (si/no): ")
if (edad < 12 or edad >= 65) and residente == "si":
    print("Entrada gratis")
else:
    print("Paga entrada")
```

Análisis paso a paso, caso A: la persona escribe 8 y si

1. Comparaciones: `8 < 12` es `True`; `"si" == "si"` es `True`. (La comparación `8 >= 65` no hace falta: el `or` ya tiene una verdadera.)
2. Paréntesis: `True or …` da `True`.
3. Cálculo final: `True and True` da `True`. Se muestra `Entrada gratis`.

Análisis paso a paso, caso B: la persona escribe 70 y no

1. Comparaciones: `70 < 12` es `False`; `70 >= 65` es `True`; `"no" == "si"` es `False`.
2. Paréntesis: `False or True` da `True`.
3. Cálculo final: `True and False` da `False`. Se muestra `Paga entrada`, porque no vive en la ciudad.

Análisis paso a paso, caso C: la persona escribe 30 y si

1. Comparaciones: `30 < 12` es `False`; `30 >= 65` es `False`; `"si" == "si"` es `True`.
2. Paréntesis: `False or False` da `False`.
3. Cálculo final: `False and True` da `False`. Se muestra `Paga entrada`, porque no cumple la edad.

Edad	Residente	Paréntesis: `edad < 12 or edad >= 65`	`residente == "si"`	Resultado final	Salida
`8`	`si`	`True`	`True`	`True`	`Entrada gratis`
`65`	`si`	`True`	`True`	`True`	`Entrada gratis`
`70`	`no`	`True`	`False`	`False`	`Paga entrada`
`30`	`si`	`False`	`True`	`False`	`Paga entrada`

> ***Idea clave:*** *aquí el grupo de alternativas va* ***al principio****. El paréntesis puede ir antes o después del* `and`*; lo importante es encerrar lo que se resuelve junto.*

Ejemplo 4: Domicilio gratis

Enunciado. Un restaurante ofrece domicilio gratis en dos situaciones: (1) la compra es de 50000 pesos o más **y** el domicilio queda a 5 kilómetros o menos; o (2) la persona es cliente premium, sin importar el valor ni la distancia. Escribe un programa que pida, en este orden, el valor de la compra (entero), la distancia en kilómetros (entero) y si es cliente premium (`si` o `no`). Si el domicilio es gratis, debe mostrar `Domicilio gratis`; si no, debe mostrar `Domicilio con costo`.

```
compra = int(input("Valor de la compra: "))
distancia = int(input("Distancia en km: "))
premium = input("¿Eres cliente premium? (si/no): ")
if (compra >= 50000 and distancia <= 5) or premium == "si":
    print("Domicilio gratis")
else:
    print("Domicilio con costo")
```

Análisis paso a paso, caso A: la persona escribe 60000, 3 y no

1. Comparaciones: `60000 >= 50000` es `True`; `3 <= 5` es `True`; `"no" == "si"` es `False`.
2. Paréntesis: `True and True` da `True`.
3. Cálculo final: `True or False` da `True`. Se muestra `Domicilio gratis`.

Análisis paso a paso, caso B: la persona escribe 60000, 8 y no

1. Comparaciones: `60000 >= 50000` es `True`; `8 <= 5` es `False`; `"no" == "si"` es `False`.
2. Paréntesis: `True and False` da `False`.
3. Cálculo final: `False or False` da `False`. Se muestra `Domicilio con costo`.

Análisis paso a paso, caso C: la persona escribe 30000, 10 y si

1. Comparaciones: `30000 >= 50000` es `False`; `10 <= 5` es `False`; `"si" == "si"` es `True`.
2. Paréntesis: `False and False` da `False`.
3. Cálculo final: `False or True` da `True`. Se muestra `Domicilio gratis`, gracias a ser cliente premium.

Compra	Distancia	Premium	Paréntesis: `compra >= 50000 and distancia <= 5`	`premium == "si"`	Resultado final	Salida
`60000`	`3`	`no`	`True`	`False`	`True`	`Domicilio gratis`
`60000`	`8`	`no`	`False`	`False`	`False`	`Domicilio con costo`
`30000`	`10`	`si`	`False`	`True`	`True`	`Domicilio gratis`
`30000`	`3`	`no`	`False`	`False`	`False`	`Domicilio con costo`

> ***Idea clave:*** *en este caso los paréntesis no cambian el resultado, porque* `and` *ya se resuelve antes que* `or`*. Aun así conviene escribirlos: muestran de un vistazo que hay* ***dos caminos*** *para obtener el domicilio gratis.*

Ejemplo 5: Los paréntesis cambian el resultado

Enunciado. Al laboratorio pueden entrar profesores **y** estudiantes, pero en ambos casos deben llevar el carné. Los visitantes no pueden entrar. Escribe un programa que pida el rol (`profesor`, `estudiante` o `visitante`, en minúsculas) y si lleva el carné (`si` o `no`), en ese orden. Si puede entrar, debe mostrar `Puedes entrar al laboratorio`; si no, debe mostrar `No puedes entrar al laboratorio`.

Versión correcta (con paréntesis):

```
rol = input("Rol (profesor/estudiante/visitante): ")
carne = input("¿Llevas el carné? (si/no): ")
if (rol == "profesor" or rol == "estudiante") and carne == "si":
    print("Puedes entrar al laboratorio")
else:
    print("No puedes entrar al laboratorio")
```

Análisis paso a paso, caso A: la persona escribe profesor y si

1. Comparaciones: `"profesor" == "profesor"` es `True`; `"si" == "si"` es `True`.
2. Paréntesis: `True or …` da `True`.
3. Cálculo final: `True and True` da `True`. Se muestra `Puedes entrar al laboratorio`.

Análisis paso a paso, caso B: la persona escribe profesor y no

1. Comparaciones: `"profesor" == "profesor"` es `True`; `"no" == "si"` es `False`.
2. Paréntesis: `True or …` da `True`.
3. Cálculo final: `True and False` da `False`. Se muestra `No puedes entrar al laboratorio`: sin carné no se entra.

Análisis paso a paso, caso C: la persona escribe visitante y si

1. Comparaciones: `"visitante" == "profesor"` es `False`; `"visitante" == "estudiante"` es `False`; `"si" == "si"` es `True`.
2. Paréntesis: `False or False` da `False`.
3. Cálculo final: `False and True` da `False`. Se muestra `No puedes entrar al laboratorio`.

Versión incorrecta (sin paréntesis), para que veas el error:

```
if rol == "profesor" or rol == "estudiante" and carne == "si":
```

Python resuelve primero el `and`, así que lee esa línea como `rol == "profesor" or (rol == "estudiante" and carne == "si")`. Eso significa que cualquier profesor entra aunque no lleve el carné.

Rol	Carné	Versión correcta (con paréntesis)	Versión incorrecta (sin paréntesis)	¿Coinciden?
`profesor`	`si`	Entra	Entra	Sí
`profesor`	`no`	**No entra**	**Entra**	**No: error**
`estudiante`	`si`	Entra	Entra	Sí
`estudiante`	`no`	No entra	No entra	Sí
`visitante`	`si`	No entra	No entra	Sí

> ***Idea clave:*** *un solo par de paréntesis puede cambiar la respuesta del programa. Ante la duda, escríbelos y haz una prueba con el caso que más te preocupa (aquí, el profesor sin carné).*

8. Condicionales anidados: una decisión dentro de otra

Un condicional anidado es un `if` escrito **dentro del bloque** de otro `if` (o de un `else`). Se usa cuando la segunda pregunta solo tiene sentido si la primera ya se cumplió.

Piensa en este ejemplo: «Si hoy hay clase, miro si ya es hora de salir; si hoy no hay clase, la hora no me importa». La pregunta sobre la hora **depende** de la primera respuesta.

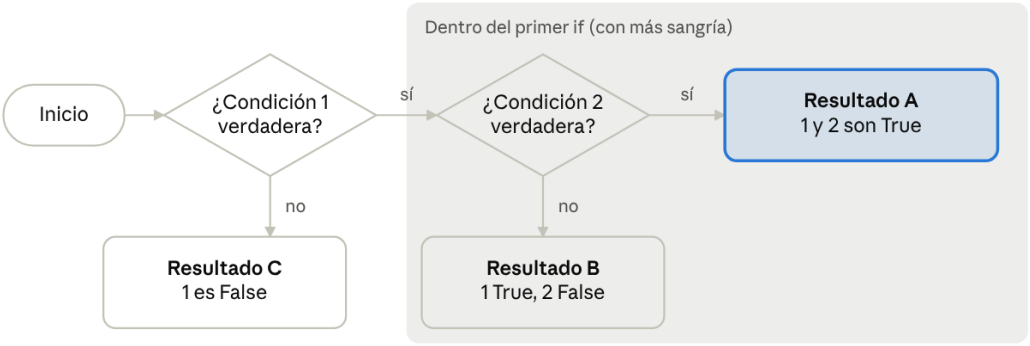
Estructura

```
if condicion1:
    # nivel 1: se ejecuta si condicion1 es True
    if condicion2:
        # nivel 2: condicion1 es True y condicion2 es True
    else:
        # nivel 2: condicion1 es True y condicion2 es False
else:
    # nivel 1: condicion1 es False (condicion2 ni se pregunta)
```

Cómo decide Python

Python hace la primera pregunta. Si es falsa, se va directo al `else` de ese nivel y **nunca** pregunta la segunda. Si es verdadera, entra a su bloque y ahí hace la segunda pregunta.

La segunda pregunta solo se hace si la primera fue verdadera



if anidado · 2 preguntas, 3 resultados

La zona sombreada es el bloque del primer `if`: todo lo que ocurre ahí tiene más sangría. Con solo dos preguntas se obtienen tres resultados distintos.

Las reglas de la sangría en el anidamiento

Nivel	Espacios a la izquierda	Qué contiene
0	0	El primer `if` y su `else`
1	4	Todo lo que depende del primer `if` (incluido el segundo `if`)
2	8	Todo lo que depende del segundo `if`

- Cada vez que entras a un bloque nuevo, sumas **4 espacios**.
- Un `else` siempre pertenece al `if` que esté **exactamente en su misma columna**.
- Cuando dudes de qué `else` pertenece a qué `if`, traza una línea vertical hacia arriba desde el `else` hasta encontrar el `if` con la misma sangría.

¿Anidar o usar `and`?

Situación	Qué conviene
Solo hay dos resultados: «se cumple todo» o «no»	`and`
Quieres un **mensaje distinto** según cuál condición falló	Anidamiento
La segunda pregunta solo existe si la primera fue verdadera	Anidamiento

Por ejemplo, con `and` un cajero solo puede decir «acceso denegado». Con anidamiento puede decir si falló la tarjeta o si falló la clave.

Cinco ejemplos analizados paso a paso

Ejemplo 1: Cajero con tarjeta y clave

Enunciado. Escribe un programa que pregunte si la persona insertó la tarjeta (`si` o `no`). Si no la insertó, debe mostrar `Inserta tu tarjeta`. Si la insertó, debe pedir la clave: si es `4321` debe mostrar `Bienvenido`; si es otra, debe mostrar `Clave incorrecta`.

```
tarjeta = input("¿Insertaste la tarjeta? (si/no): ")
if tarjeta == "si":
    clave = input("Escribe tu clave: ")
    if clave == "4321":
        print("Bienvenido")
    else:
        print("Clave incorrecta")
else:
    print("Inserta tu tarjeta")
```

Análisis paso a paso, caso A: la persona escribe si y luego 4321

1. Línea 1: `tarjeta` guarda `"si"`.
2. Línea 2 (nivel 0): Python evalúa `"si" == "si"`. Es `True`, así que entra al bloque con 4 espacios.
3. Línea 3 (nivel 1): `clave` guarda `"4321"`. Esta pregunta **solo se hizo** porque la tarjeta fue insertada.
4. Línea 4 (nivel 1): Python evalúa `"4321" == "4321"`. Es `True`, así que entra al bloque con 8 espacios.
5. Línea 5 (nivel 2): se muestra `Bienvenido`.
6. Las líneas 6, 7, 8 y 9 se saltan, porque pertenecen a los `else`.

Análisis paso a paso, caso B: la persona escribe si y luego 1111

1. Línea 2: `"si" == "si"` es `True`; entra al nivel 1.
2. Línea 3: `clave` guarda `"1111"`.
3. Línea 4: `"1111" == "4321"` es `False`. El bloque de la línea 5 se salta.
4. Línea 7: se ejecuta el `else` de la **línea 6**, que tiene 4 espacios, y se muestra `Clave incorrecta`.
5. Línea 9 no se ejecuta, porque ese `else` pertenece al primer `if`, que fue verdadero.

Análisis paso a paso, caso C: la persona escribe no

1. Línea 2: `"no" == "si"` es `False`. Python salta todo el bloque de las líneas 3 a 7.
2. Línea 9: se ejecuta el `else` del primer `if` y se muestra `Inserta tu tarjeta`.
3. **Nunca se pidió la clave**, porque esa pregunta no tenía sentido sin tarjeta.

Tarjeta	Clave	Primera pregunta	Segunda pregunta	Salida
`si`	`4321`	`True`	`True`	`Bienvenido`
`si`	`1111`	`True`	`False`	`Clave incorrecta`
`no`	*(no se pide)*	`False`	*(no se hace)*	`Inserta tu tarjeta`

> ***Idea clave:*** *hay **tres** salidas posibles con solo dos preguntas, y cada una dice exactamente qué pasó.*

Ejemplo 2: Compra solo para mayores de edad

Enunciado. Escribe un programa que pida la edad (entero). Si la persona es menor de 18 años, debe mostrar `Eres menor de edad` y no pedir nada más. Si tiene 18 años o más, debe pedir el dinero que tiene (entero) para pagar una compra de 20000 pesos: si tiene 20000 o más, debe mostrar `Compra realizada`; si tiene menos, debe mostrar `Dinero insuficiente`.

```
edad = int(input("Edad: "))
if edad >= 18:
    dinero = int(input("¿Cuánto dinero tienes? "))
    if dinero >= 20000:
        print("Compra realizada")
    else:
        print("Dinero insuficiente")
else:
    print("Eres menor de edad")
```

Análisis paso a paso, caso A: la persona escribe 25 y luego 30000

1. Línea 1: `edad` guarda 25.
2. Línea 2: `25 >= 18` es `True`; entra al nivel 1.
3. Línea 3: `dinero` guarda 30000.
4. Línea 4: `30000 >= 20000` es `True`; entra al nivel 2.
5. Línea 5: se muestra `Compra realizada`.

Análisis paso a paso, caso B: la persona escribe 25 y luego 5000

1. Línea 2: `25 >= 18` es `True`; entra al nivel 1.
2. Línea 3: `dinero` guarda 5000.
3. Línea 4: `5000 >= 20000` es `False`; se salta la línea 5.
4. Línea 7: se muestra `Dinero insuficiente`.

Análisis paso a paso, caso C: la persona escribe 15

1. Línea 2: `15 >= 18` es `False`; se salta todo el bloque de las líneas 3 a 7.
2. Línea 9: se muestra `Eres menor de edad`. El programa nunca pregunta por el dinero.

Edad	Dinero	Primera pregunta	Segunda pregunta	Salida
`25`	`30000`	`True`	`True`	`Compra realizada`
`25`	`5000`	`True`	`False`	`Dinero insuficiente`
`18`	`20000`	`True`	`True`	`Compra realizada`
`15`	*(no se pide)*	`False`	*(no se hace)*	`Eres menor de edad`

> ***Idea clave:*** *la pregunta sobre el dinero solo se hace a las personas mayores de edad. Eso es lo que hace que el anidamiento sea la mejor opción.*

Ejemplo 3: Clasificar una nota (tres niveles)

Enunciado. Escribe un programa que pida una nota (número que puede tener decimales). Si la nota no está entre 0 y 5 (ambos incluidos), debe mostrar `Nota no válida`. Si es válida y es menor que 3.0, debe

mostrar `Reprobado`. Si es válida y es 3.0 o más, debe mostrar `Aprobado`, salvo que sea 4.5 o más: en ese caso debe mostrar `Aprobado con excelencia`.

```
nota = float(input("Escribe la nota (0 a 5): "))
if nota >= 0 and nota <= 5:
    if nota >= 3.0:
        if nota >= 4.5:
            print("Aprobado con excelencia")
        else:
            print("Aprobado")
    else:
        print("Reprobado")
else:
    print("Nota no válida")
```

Análisis paso a paso, caso A: la persona escribe 4.8

1. Línea 1: `nota` guarda 4.8.
2. Línea 2 (nivel 0): `4.8 >= 0 and 4.8 <= 5` es `True and True`, o sea `True`. Entra al nivel 1.
3. Línea 3 (nivel 1): `4.8 >= 3.0` es `True`. Entra al nivel 2.
4. Línea 4 (nivel 2): `4.8 >= 4.5` es `True`. Entra al nivel 3.
5. Línea 5: se muestra `Aprobado con excelencia`. Todos los `else` se saltan.

Análisis paso a paso, caso B: la persona escribe 3.5

1. Línea 2: la nota es válida (`True and True`). Entra al nivel 1.
2. Línea 3: `3.5 >= 3.0` es `True`. Entra al nivel 2.
3. Línea 4: `3.5 >= 4.5` es `False`. Se salta la línea 5.
4. Línea 7: se ejecuta el `else` de la línea 6 y se muestra `Aprobado`.

Análisis paso a paso, caso C: la persona escribe 2.0

1. Línea 2: la nota es válida. Entra al nivel 1.
2. Línea 3: `2.0 >= 3.0` es `False`. Se salta todo el bloque de las líneas 4 a 7.
3. Línea 9: se ejecuta el `else` de la línea 8 y se muestra `Reprobado`.

Análisis paso a paso, caso D: la persona escribe 7

1. Línea 2: `7.0 >= 0` es `True`, pero `7.0 <= 5` es `False`. El `and` da `False`.
2. Python salta todo el bloque de las líneas 3 a 9.
3. Línea 11: se ejecuta el `else` de la línea 10 y se muestra `Nota no válida`.

Nota	¿Válida? (nivel 0)	`nota >= 3.0` (nivel 1)	`nota >= 4.5` (nivel 2)	Salida
`4.8`	`True`	`True`	`True`	`Aprobado con excelencia`
`4.5`	`True`	`True`	`True`	`Aprobado con excelencia`
`3.5`	`True`	`True`	`False`	`Aprobado`
`3.0`	`True`	`True`	`False`	`Aprobado`

Nota	¿Válida? (nivel 0)	`nota >= 3.0` (nivel 1)	`nota >= 4.5` (nivel 2)	Salida
`2.0`	`True`	`False`	*(no se hace)*	`Reprobado`
`7`	`False`	*(no se hace)*	*(no se hace)*	`Nota no válida`

> ***Idea clave:*** *puedes anidar tantos niveles como necesites, y cada nivel puede usar* `and` *u* `or`*. Primero se descartan los datos inválidos y después se clasifica.*

Ejemplo 4: Positivo, negativo o cero

Enunciado. Escribe un programa que pida un número entero. Si el número es negativo, debe mostrar `Es negativo`. Si no es negativo, debe distinguir: si es exactamente 0, debe mostrar `Es cero`; en cualquier otro caso, debe mostrar `Es positivo`.

```
numero = int(input("Escribe un número entero: "))
if numero >= 0:
    if numero == 0:
        print("Es cero")
    else:
        print("Es positivo")
else:
    print("Es negativo")
```

Análisis paso a paso, caso A: la persona escribe 9

1. Línea 1: `numero` guarda 9.
2. Línea 2 (nivel 0): `9 >= 0` es `True`. Entra al nivel 1.
3. Línea 3 (nivel 1): `9 == 0` es `False`. Se salta la línea 4.
4. Línea 6: se ejecuta el `else` de la línea 5 y se muestra `Es positivo`.

Análisis paso a paso, caso B: la persona escribe 0

1. Línea 2: `0 >= 0` es `True`. Entra al nivel 1.
2. Línea 3: `0 == 0` es `True`.
3. Línea 4: se muestra `Es cero`.

Análisis paso a paso, caso C: la persona escribe -4

1. Línea 2: `-4 >= 0` es `False`. Se salta todo el bloque de las líneas 3 a 6.
2. Línea 8: se ejecuta el `else` de la línea 7 y se muestra `Es negativo`.

Número	`numero >= 0` (nivel 0)	`numero == 0` (nivel 1)	Salida
`9`	`True`	`False`	`Es positivo`
`0`	`True`	`True`	`Es cero`
`-4`	`False`	*(no se hace)*	`Es negativo`

> ***Idea clave:*** *no hace falta preguntar «¿es mayor que 0?». Si el programa llegó al nivel 1, ya se sabe que el número es 0 o más; si además no es 0, solo puede ser positivo.*

Ejemplo 5: Solicitud de crédito (tres preguntas encadenadas)

Enunciado. Escribe un programa que decida si se aprueba un crédito. Primero pide la edad (entero). Si es menor de 18, debe mostrar `No cumples la edad mínima`. Si tiene 18 años o más, debe preguntar si tiene empleo (`si` o `no`). Si no tiene empleo, debe mostrar `Necesitas un empleo para solicitar el crédito`. Si tiene empleo, debe pedir los ingresos mensuales en pesos (entero): con 2000000 o más debe mostrar `Crédito aprobado`; con menos, debe mostrar `Ingresos insuficientes`.

```
edad = int(input("Edad: "))
if edad >= 18:
    empleo = input("¿Tienes empleo? (si/no): ")
    if empleo == "si":
        ingresos = int(input("Ingresos mensuales en pesos: "))
        if ingresos >= 2000000:
            print("Crédito aprobado")
        else:
            print("Ingresos insuficientes")
    else:
        print("Necesitas un empleo para solicitar el crédito")
else:
    print("No cumples la edad mínima")
```

Análisis paso a paso, caso A: la persona escribe 30, si y 3000000

1. Línea 1: `edad` guarda 30.
2. Línea 2 (nivel 0): `30 >= 18` es `True`. Entra al nivel 1.
3. Línea 3 (nivel 1): `empleo` guarda `"si"`.
4. Línea 4 (nivel 1): `"si" == "si"` es `True`. Entra al nivel 2.
5. Línea 5 (nivel 2): `ingresos` guarda 3000000.
6. Línea 6 (nivel 2): `3000000 >= 2000000` es `True`. Entra al nivel 3.
7. Línea 7: se muestra `Crédito aprobado`.

Análisis paso a paso, caso B: la persona escribe 30, si y 1500000

1. Líneas 2 a 5: igual que el caso A; `ingresos` guarda 1500000.
2. Línea 6: `1500000 >= 2000000` es `False`. Se salta la línea 7.
3. Línea 9: se ejecuta el `else` de la línea 8 y se muestra `Ingresos insuficientes`.

Análisis paso a paso, caso C: la persona escribe 30 y no

1. Línea 2: `30 >= 18` es `True`. Entra al nivel 1.
2. Línea 3: `empleo` guarda `"no"`.
3. Línea 4: `"no" == "si"` es `False`. Se salta todo el bloque de las líneas 5 a 9.
4. Línea 11: se ejecuta el `else` de la línea 10 y se muestra `Necesitas un empleo para solicitar el crédito`.
5. **Nunca se pidieron los ingresos.**

Análisis paso a paso, caso D: la persona escribe 16

1. Línea 2: `16 >= 18` es `False`. Se salta todo el bloque de las líneas 3 a 11.
2. Línea 13: se ejecuta el `else` de la línea 12 y se muestra `No cumples la edad mínima`.

Edad	Empleo	Ingresos	Pregunta 1	Pregunta 2	Pregunta 3	Salida
`30`	`si`	`3000000`	`True`	`True`	`True`	`Crédito aprobado`
`30`	`si`	`1500000`	`True`	`True`	`False`	`Ingresos insuficientes`
`30`	`no`	*(no se pide)*	`True`	`False`	*(no se hace)*	`Necesitas un empleo para solicitar el crédito`
`16`	*(no se pide)*	*(no se pide)*	`False`	*(no se hace)*	*(no se hace)*	`No cumples la edad mínima`

> ***Idea clave:*** *cada pregunta aparece solo si las anteriores se cumplieron. Por eso el programa nunca le pide los ingresos a quien no tiene empleo.*